

Heterogeneous E/PD Disaggregation in Dynamo Framework

Daniel Socek, Sergey Plotnikov, Pallavi Jaini, and Brian Liu

Intel Corporation, 2200 Mission College Blvd., Santa Clara, CA
95054-1549, USA

{daniel.soczek, sergey.plotnikov, pallavi.jaini,
brian.liu}@intel.com

May 2026

Abstract

We analyze a heterogeneous encoder/prefill-decode (E/PD) disaggregated serving architecture for multimodal large language model inference within the NVIDIA Dynamo framework. Our approach decouples visual encoding from the language-model prefill and decode stages, enabling Vision Transformer (ViT) workloads to execute on low-cost GPUs while reserving high-end GPUs for language-model execution. We evaluate system behavior under light, moderate, and heavy load conditions and demonstrate that the benefits of disaggregation vary significantly across these regimes. Under light load, disaggregation provides no gain and slightly increases TTFT due to communication overhead. Under moderate load, TTFT improves through overlap of embedding computation with prefill execution, with the magnitude of benefit governed by the relative encoding and prefill times. Under heavy load, the dominant benefit is indirect: offloading ViT computation to dedicated E-workers reduces queue length on the PD-worker, which in turn substantially lowers TTFT. Experiments on Qwen3-VL-32B-Instruct (FP8) show that four Intel Arc B60 E-workers outperform an aggregated NVIDIA H200 baseline at moderate and high request rates where the aggregated configuration exhibits severe latency growth. Overall, heterogeneous E/PD disaggregation provides a favorable cost-performance trade-off by extending serving scalability through inexpensive encoder resources.

1 Introduction

The rapid adoption of large language models (LLMs) in production has created strong pressure to reduce inference cost while maintaining low latency. A key

observation is that multi-modal LLM inference can be decomposed into three stages, each with distinct compute, memory, and communication characteristics:

1. *Encode (E)*: A modality-specific encoder transforms raw inputs into embedding vectors. For images and video, this is typically a Vision Transformer (ViT). These encoders are substantially smaller than the language-model backbone and can run efficiently on lower-cost accelerators, but remain too compute-intensive for typical edge-class CPUs at serving latency targets.
2. *Prefill (P)*: The language model processes the full prompt in a forward pass, constructing the key-value (KV) cache. This stage is both compute- and memory-intensive, as it requires full attention over the entire prompt and dominates the initial latency.
3. *Decode (D)*: Tokens are generated iteratively using the KV cache. Each step performs a forward pass over a single token, making this stage primarily memory-bandwidth-bound and sensitive to KV cache access patterns.

1.1 Background and Related Work

Recent work has shown that routing the prefill and decode phases to dedicated hardware pools can substantially improve serving efficiency. Prefill/decode (P/D) disaggregation separates the compute-heavy prefill phase from the memory-bandwidth-sensitive decode phase, enabling each stage to be provisioned and scheduled independently [1, 2]. This idea has recently been extended to multi-modal serving through encoder/prefill/decode (E/P/D) disaggregation, where visual encoding is separated from language-model execution [3, 4].

Open-source serving systems such as vLLM and SGLang have begun incorporating support for encoder disaggregation, reflecting growing practical interest in decoupling visual encoding from language-model execution [5, 6]. In particular, [5] introduces encoder offloading by executing the vision encoder externally and integrating precomputed embeddings into its native inference runtime, while [6] supports a more complete EPD-style execution model with explicit separation of encoder, prefill, and decode stages. The NVIDIA Dynamo framework [7], which supports both vLLM and SGLang backends, provides the system infrastructure used in this work: a modular LLM serving stack supporting disaggregated workers and high-speed data transfer via NIXL.

In this paper, we extend E/PD disaggregation within Dynamo using the vLLM backend to a *heterogeneous* setting, where low-end GPUs are used for E-workers that execute only the encoding stage, while high-end GPUs are used for PD-workers that perform both prefill and decode. In particular, our evaluation uses Intel Arc Pro B60 GPUs for E-workers and NVIDIA H200 GPUs for PD-workers. We analyze the resulting latency, throughput, and cost trade-offs.

2 Dynamo Architecture Overview

Dynamo is structured around loosely-coupled, independently-scalable worker processes connected by a high-throughput data plane. In this paper, we focus on the following four components:

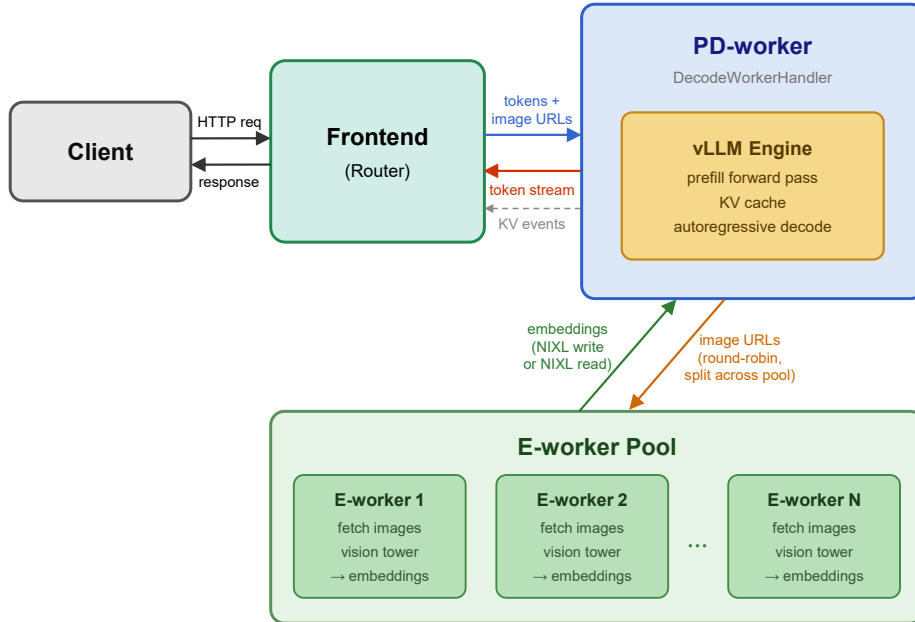


Figure 1: Architecture of the heterogeneous E/PD disaggregated serving system: The frontend router dispatches requests to the PD worker, which offloads vision encoding to a pool of E workers. Image URLs are sent to E workers over the Dynamo transport (TCP/NATS), and the resulting embeddings are transferred back to the PD worker via NIXL Connect.

- **Frontend router.** Accepts HTTP/gRPC requests and dispatches them to workers. The router supports multiple load-balancing strategies, including a KV-aware mode that tracks per-worker KV-cache occupancy to make placement decisions that reduce redundant prefill work.
- **E-worker.** Runs the modality encoder on a dedicated accelerator. Upon receiving image URLs (e.g., links or base64 data) from the PD-worker, it fetches the images, runs the vision tower, and ships the resulting embedding tensors back via NIXL. The E-worker pool scales out simply by adding more nodes, since individual media items are independent (e.g., different images in a batch).
- **PD-worker.** A deployment-level label for a worker that runs both the prefill and decode phases. In the E/PD topology studied here, embeddings

produced by the E-worker are made available to the PD-worker, which runs the prefill forward pass to populate the KV cache and then performs autoregressive decoding using a single vLLM engine instance.

The PD-worker partitions the media items of each multimodal request into encoder batches and dispatches them to the E-worker pool using round-robin scheduling (in Dynamo 1.0.0). For request i with n media items and N_E E-workers, the default encoder batch size is

$$B_i = \max \left(1, \left\lfloor \frac{n}{N_E} \right\rfloor \right).$$

Each encoder batch is assigned to E-worker $e = c \bmod N_E$, where c is the PD-worker’s round-robin dispatch counter. Thus, large requests are split across E-workers; for example, 20 images and four E-workers produce 4 batches of 5 images.

In a fully disaggregated E/P/D topology, the PD role is split across two separate GPU workers. The *P-worker* orchestrates encoding offload, fuses the returned embeddings with the prompt tokens, and runs the prefill forward pass to compute the KV cache. The *D-worker* receives the KV cache from the P-worker via NIXL and performs autoregressive decoding. This separation avoids co-locating prefill and decode on the same GPU, reducing pipeline interference between the two phases. It also allows the encode, prefill, and decode worker pools to scale independently.

- **NIXL data-transfer layer.** Provides kernel-bypass RDMA (GPUDirect / InfiniBand / RoCE) transport for moving embedding tensors between E-workers and PD-workers. Two transfer modes are supported: *NIXL write* (E-worker pushes data to the receiver’s memory) and *NIXL read* (PD-worker pulls data from the E-worker). In fully disaggregated P/D topologies NIXL also carries KV-cache slices from prefill to decode workers. The choice of transport has a measurable impact on TTFT (Section 4.1).

Dynamo supports a range of disaggregation topologies; the ones relevant to this paper are summarized in Table 1.

Table 1: Multimodal disaggregation topologies supported by Dynamo.

Topology	Workers	Main worker	Request flow
Aggregated	$n \times \text{Agg}$	<code>DecodeWorkerHandler</code>	Frontend $\rightarrow$ Agg $\rightarrow$ Client
E/PD	$n \times \text{E} + m \times \text{PD}$	<code>DecodeWorkerHandler</code>	Frontend $\rightarrow$ PD $\xrightarrow{\text{URLs}}$ E $\xrightarrow{\text{emb}}$ PD $\rightarrow$ Client
E/P/D	$n \times \text{E} + m \times \text{P} + k \times \text{D}$	<code>PrefillWorkerHandler</code>	Frontend $\rightarrow$ P $\xrightarrow{\text{URLs}}$ E $\xrightarrow{\text{emb}}$ P $\xrightarrow{\text{KV}}$ Router $\rightarrow$ D $\rightarrow$ Client

This paper focuses on the **E/PD** topology: a single aggregated PD-worker offloads vision encoding to a pool of Intel Arc B60 E-workers, keeping the high-end GPU free for prefill and decode.

3 E/PD Control Flow

Understanding the end-to-end control and data flow is essential for interpreting the performance results in Section 4. We trace a single inference request from client to first token and back, focusing on the E/PD scenario (separate encode worker, combined prefill+decode worker) running multimodal vision language model with multi-image requests.

Requests originate from `vllm bench`, a benchmark client built on the vLLM framework. The client issues an HTTP request containing the prompt text, input media (base64-encoded bitstreams as data URLs), and generation parameters (e.g., maximum new tokens, sampling temperature). The client records the timestamp at request submission and the arrival time of the first generated token, enabling direct measurement of *time to first token* (TTFT).

The *frontend* is a Rust-based component that receives the client request, applies the chat template, tokenizes the prompt on CPU, extracts image URLs from the multimodal content, and dispatches the preprocessed request to the PD-worker via the Dynamo message bus (NATS or TCP). The frontend performs no GPU inference or vision encoding; those model execution stages are handled by the downstream workers

Upon receiving a multimodal request, the PD-worker partitions the image URLs into encoder batches and serializes each batch into a JSON payload dispatched to available E-workers over the Dynamo message bus using round-robin load balancing. The PD-worker then asynchronously awaits the encode results and retrieves the resulting visual embeddings. In NIXL read mode, it uses the returned transfer metadata to initiate an RDMA read via NIXL, transferring the embedding tensors into local memory through a registered NIXL descriptor.

The E-worker receives the JSON request containing image URLs and runs the following steps:

1. Decodes the base64-encoded image data on CPU (asynchronously, multi-threaded) and runs it through the image processor to produce pixel tensors.
2. Feeds the pixel tensors into the vision encoder on GPU, producing visual embeddings whose sequence length depends on the image resolution, patch size, and model-specific preprocessing.
3. Registers the embedding tensor in a NIXL-readable GPU buffer and returns a lightweight JSON descriptor (handle) to the PD-worker. The PD-worker subsequently performs an RDMA read to pull the tensor.

Once the *PD-worker* has retrieved the visual embedding tensor from the E-worker, it:

1. Assembles the `multi_modal_data` dictionary (accumulating embeddings and grid metadata for all images) and passes it alongside the tokenized prompt to the vLLM engine.
2. Runs the prefill forward pass, executing the language-model transformer layers over the full prompt and populating the KV cache.

3. Completes the prefill phase and transitions the request into autoregressive generation.
4. Enters the autoregressive decode loop, generating one token per step until an EOS token or the maximum length is reached.

Generated tokens are streamed back to the client through the PD-worker $\rightarrow$ frontend path as they become available.

4 Performance Analysis

This section evaluates the performance characteristics of disaggregated E/PD serving with NVIDIA Dynamo. We focus on three aspects of system behavior: the overhead of visual embedding transfer between encode and prefill-decode workers, the key serving metrics used to evaluate end-to-end performance, and the throughput-latency tradeoffs observed across deployment configurations.

4.1 Embeddings Transfer

Visual embeddings transfer is smaller than KV-cache transfer but remains non-negligible for image-dense and video workloads. The multimodal payload can reach hundreds of MB per request, and because prefill cannot begin until these embeddings are available, transfer latency directly contributes to end-to-end latency.

We evaluate a workload with 20 images at resolution 480×854 . For the Qwen3-VL-32B-Instruct model used in this evaluation, each image produces 405 visual tokens with embedding dimension 20,480 in BF16 precision, giving

$$405 \times 20,480 \times 2 \text{ bytes} = 16.6 \text{ MB} \quad (1)$$

per image, or approximately 332 MB per request.

We compare the TCP and RDMA transfer backends in NIXL using the hardware configuration described in Section 4.3. In our TCP configuration, transfers are staged through host memory and the kernel networking stack:

$$\text{Device} \rightarrow \text{Host} \rightarrow \text{Kernel} \rightarrow \text{NIC} \rightarrow \text{Kernel} \rightarrow \text{Host} \rightarrow \text{Device}. \quad (2)$$

RDMA provides kernel-bypass communication over registered memory regions [8]. In our setup, we use RDMA-backed NIXL to transfer embeddings from XPU memory to CUDA GPU memory using registered device buffers, avoiding expensive CPU memcpy operations through host memory.

We evaluate both NIXL transfer primitives. With `nixl_write`, the PD-worker pre-allocates and exposes destination buffers, and the E-worker pushes embeddings into those buffers once encoding completes. With `nixl_read`, the E-worker exposes the produced embedding buffer, and the PD-worker initiates an on-demand pull into its local memory. Thus, write-based transfer shifts buffer

Table 2: Median embedding transfer latency for a 332 MB payload.

Transport	Primitive	Transfer latency (ms)
TCP	NIXL Read	479
TCP	NIXL Write	323
RDMA	NIXL Read	80
RDMA	NIXL Write	22

provisioning to the receiver ahead of time, whereas read-based transfer gives the receiver control over when data is fetched.

We measure median transfer latency over 32 requests, with request arrivals generated by a Poisson process at an average rate of 2.0 requests per second (RPS). Each request transfers approximately 332 MB of embeddings from the XPU-based E-worker to the CUDA-based PD-worker.

Table 2 shows that RDMA achieves substantially lower transfer latency than TCP. The TCP based transfer is dominated by host staging and kernel-mediated data movement, reaching only ~ 0.7 GB/s for `nixl_read` and ~ 1 GB/s for `nixl_write`. RDMA improves throughput to ~ 4 GB/s and ~ 15 GB/s, respectively.

In this NIXL transfer benchmark, `nixl_write` provides lower raw embedding-transfer latency than `nixl_read`. While `nixl_write` requires the PD-worker to provision destination buffers ahead of time, introducing additional memory-management complexity in the serving pipeline, our end-to-end experiments also showed better overall TTFT behavior with the `nixl_write` path. We therefore use RDMA-backed `nixl_write` for the full serving performance experiments in Section 4.

4.2 Performance Metrics

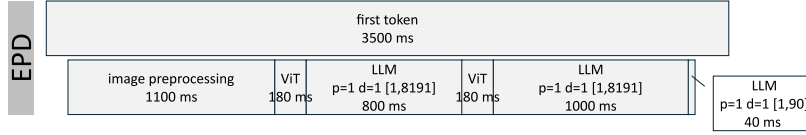
4.2.1 Time to First Token

Time to First Token (TTFT) is an important metric for interactive LLM serving and the focal point of this paper. It measures the elapsed time from request submission to the delivery of the first generated token. In this section, we discuss how E/PD disaggregation can improve TTFT. We analyze three workload types and the impact of disaggregation on each of them.

Light workload This is the case where request processing time is comparable to the time interval between request submissions. Only a few requests are processed in the system concurrently.

In this case, for the usual control flow: `encode` $\rightarrow$ `prefill` $\rightarrow$ `decode`, it follows that low-end hardware cannot improve TTFT because it cannot compute embeddings faster than a high-end GPU.

Let us examine the test results. Here and below in this section we compare the aggregated case to the disaggregated case. The aggregated case runs an EPD



(a) Aggregated case, EPD worker

(b) Disaggregated case, E and PD workers

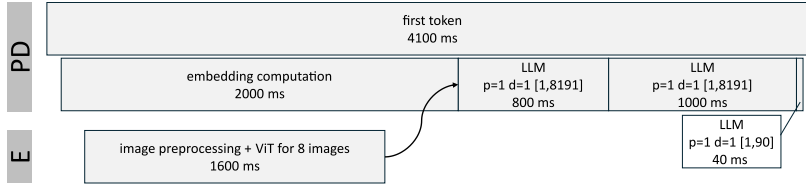


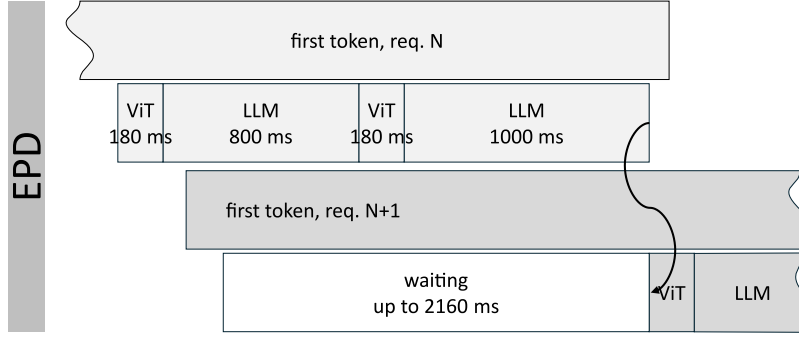
Figure 2: Traces for aggregated and disaggregated cases for light workload.”first token” - event corresponding to first token computation on EPD or PD workers; ”image preprocessing” is model specific preprocessing of input image, including decoding, resizing etc.; ”ViT” - embedding computation; ”LLM” - prefill and decoding computation; ”pre=1, dec=1” number of prefill and decoding requests on current LLM step, aka batch; ”tk [1, 8191]” - number of tokens for each of the request in the batch; ”160 ms” - event duration, wall clock

worker on a high-end GPU. The disaggregated case runs an E worker and a PD worker on two high-end GPUs of the same type. Note that the disaggregated case intentionally uses the same GPU type for the E worker to isolate the inherent cost of disaggregation. If no benefit is observed on high-end hardware, low-end alternatives need not be considered.

On Figure 2, traces for one of the requests in the middle of the test run are shown for the aggregated and disaggregated cases. As can be seen, embedding computation takes about the same time, but the overall TTFT is larger in the disaggregated case due to communication overhead. Since even a high-end E worker adds overhead without improving TTFT, moving the E worker to a low-end GPU would only make matters worse by also increasing embedding computation time.

Moderate workload In this case, request processing time is much longer than the time interval between request submissions. Many requests are processed at the same time, but the system still has plenty of resources—memory and compute.

In this case, disaggregation provides clear but still moderate performance



(a) Aggregated case, EPD worker

(b) Disaggregated case, PD worker

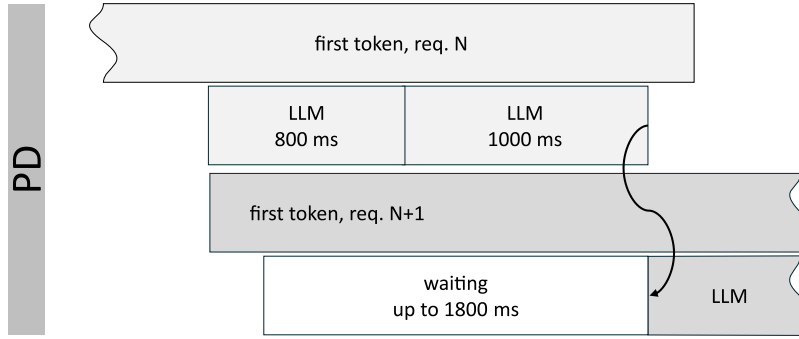


Figure 3: Traces for aggregated and disaggregated cases for moderate workload. E worker is not shown.

benefits by running embedding computation in parallel with prefill. Note that benefits are visible only when encoding workload is waiting to be executed, which happens only when prefill steps are long (on the order of seconds). For short prefill workloads (a fraction of a second), the benefits of offloading embedding computation are usually negligible. There is also no significant reduction in compute or memory pressure, because for most practical cases the ViT is a relatively small model compared to the LLM.

In other words, disaggregation helps when a new request arrives while the previous one is still in the prefill stage and the system cannot immediately start embedding computation, having to wait until prefill finishes. If the system is in the decode stage when a new request arrives, there is no benefit. Decode steps are usually an OOM shorter than prefill, and ViT offloading has negligible impact on TTFT.

Let us examine the test results (Figure 3). This is a cut from the middle of the processing. On the trace below, request $N+1$ arrives while the mixed prefill-decode batch for request N is still running. For media-heavy workloads, prefill may take a long time compared to decode. On the trace below, it requires two ViT and two LLM steps with a total duration of 2.16 seconds.

The timing values shown are from an actual test run and are given for illustrative purposes only; a detailed discussion of performance results with exact system configuration is provided in subsection 4.3.

In the aggregated case, the ViT step waits until the previous request’s prefill is done, increasing TTFT. In the disaggregated case, the $N+1$ embedding computation runs in parallel with the N th prefill, reducing wait time and overall TTFT.

The trace has been captured on a homogeneous setup with two high-end GPUs, but the benefits are also present with a low-end E worker GPU. We can reason about the expected benefit of disaggregating the E worker to a low-end GPU for requests that arrive during the prefill stage.

Let T_E^{low} denote the embedding computation time on the low-end GPU (including communication overhead), T_E^{high} the embedding computation time on the high-end GPU, and T_P the prefill duration. Assume a request arrives at a uniformly random point during prefill, so the remaining prefill time R is uniformly distributed on $[0, T_P]$. In the aggregated case, the request must wait for prefill to finish before embedding can start, giving a cost of $R + T_E^{\text{high}}$. In the disaggregated case, embedding starts immediately on the E worker, and the PD worker becomes available after $\max(R, T_E^{\text{low}})$, giving a cost of $\max(R, T_E^{\text{low}})$. The per-request benefit is therefore $R + T_E^{\text{high}} - \max(R, T_E^{\text{low}})$. When $R \geq T_E^{\text{low}}$, the embedding is fully hidden and the benefit equals T_E^{high} . When $R < T_E^{\text{low}}$, disaggregation actually hurts because the low-end E worker has not yet finished by the time prefill completes. Taking the expectation over R , the expected benefit is:

$$E[\text{benefit}] = T_E^{\text{high}} - \frac{(T_E^{\text{low}})^2}{2T_P} \quad (3)$$

This quantity is positive as long as $T_E^{\text{low}} < \sqrt{2T_P \cdot T_E^{\text{high}}}$.

However, this analysis applies only to requests that arrive during the prefill stage and experience contention. Requests that arrive during the decode stage do not benefit from disaggregation and instead incur both communication overhead and slower embedding computation on the low-end GPU. The overall impact therefore depends on many additional factors—request arrival rate, prefill and decode durations, and the probability of prefill contention—making a closed-form threshold impractical. In practice, the tradeoffs must be evaluated empirically for each specific workload and hardware configuration.

Heavy workload This case is somewhat similar to the moderate workload. In both, request processing time is much longer than the time interval between

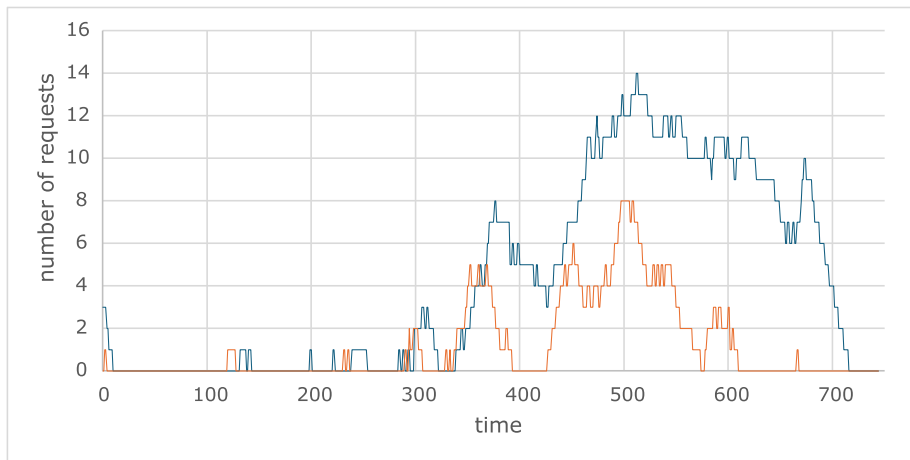


Figure 4: Queue length for aggregated (dark blue) and disaggregated (orange) cases.

request submissions, and many requests are processed concurrently. However, in contrast to the moderate workload, the system is under high pressure on compute and memory resources.

Before discussing the impact of disaggregation on performance, we must address one important factor that drastically affects performance under heavy workload and must be eliminated to correctly analyze disaggregation impact—**request preemption** caused by KV cache pressure. If this event occurs, system performance drops so severely that any other performance optimizations become meaningless.

When the number of concurrently running requests is large enough, vLLM runs out of free KV cache blocks and must preempt one of the running requests to continue decoding the rest. The preempted request is placed in the waiting queue and all its KV cache blocks are freed.

This leads to severe performance degradation for the preempted request in particular and also increases overall system workload, because all KV cache entries must be recomputed when the request is scheduled back for execution. Specifically, all prefill tokens must be reprocessed in full, effectively doubling the prefill workload. Previously generated decode tokens must also have their KV cache values recomputed, though this is done in a single LLM step and represents significantly less work than the original token-by-token decode. In the worst case, a request may be preempted multiple times, further multiplying the prefill recomputation cost.

The simplest indication of preemption is KV cache usage: if it reaches and stays close to 100%, there is a high probability that preemption is occurring and performance is significantly degraded. To mitigate this, the `--max-num-seqs` parameter can be used to limit the number of concurrently running requests in vLLM. While a sufficiently conservative value makes preemption unlikely, it



Figure 5: per request TTFT for aggregated (dark blue) and disaggregated (orange) cases

cannot be ruled out entirely because per-request KV cache consumption varies at runtime. The appropriate value can be determined experimentally and depends on model size, prompt length, and image sizes.

After eliminating request preemption, we can examine how disaggregation affects TTFT in this regime. Once vLLM reaches the specified concurrency limit, additional requests are placed in a queue. The time a request spends in the queue adds directly to its TTFT, significantly increasing it.

Disaggregation reduces overall system workload on the high-end GPU by moving embedding computation to a dedicated worker. The time that was previously spent computing embeddings is now available for the currently running requests—performing prefill and decode steps faster. This in turn reduces request processing time, which allows queued requests to be admitted sooner, reducing queue length and ultimately TTFT.

It is important to note that the TTFT reduction is not directly caused by parallel execution of ViT workloads. Rather, the primary mechanism is indirect: offloading ViT computation reduces queue size, allowing requests to spend significantly less time waiting. While the ViT offload itself provides only a relatively small direct gain (fractions of a second), the cumulative effect of reduced queue size results in significant TTFT improvements of several seconds.

Let us examine the actual queue length from one of the tests (Figure 4). The values shown are from an actual test run and are given for illustrative purposes only; a detailed discussion of performance results is provided in subsection 4.3.

As can be seen, disaggregation decreases the maximum queue length from 14 to 8 requests, which in turn reduces median TTFT from 18 to 9 seconds. While this improvement is valuable in itself, the practical benefit is even greater: with a lower TTFT per request, the system can handle a higher request rate while still meeting the TTFT SLA.

It is also instructive to examine how TTFT changes for each request during system warm-up and subsequent execution (Figure 5). This is the same run as the previous chart. Note that the X axes differ: the first chart uses time, while the second uses request number. Because requests are submitted at random times, the charts are not directly comparable.

As can be seen, the disaggregated case is a clear winner. At the beginning, when the number of concurrently running requests is relatively small, it exhibits lower TTFT due to de-serialization of the encoding stage (see the moderate workload analysis above). Then, starting at request number 40—which represents the maximum number of concurrently running requests configured for vLLM in this run—TTFT begins to grow as requests are queued. In the aggregated case, the queue grows larger and delays become significantly greater. This effect is especially noticeable around request number 100, where the number of queued requests reaches its maximum.

4.2.2 Time Per Output Token

Time Per Output Token (TPOT) measures the average time between consecutive generated tokens during the decode phase. While TTFT captures the responsiveness of the system at the start of a request, TPOT determines the sustained generation speed experienced by the user throughout the response. We analyze how E/PD disaggregation affects TPOT across the same three workload types.

Light workload When few requests are processed concurrently, the GPU is rarely interrupted by embedding computation from other requests during the decode phase. ViT invocations are infrequent and their blocking effect is negligible. In this regime, disaggregation has no meaningful impact on TPOT.

Moderate workload As concurrency increases, ViT invocations from newly arriving requests begin to interrupt decode steps on the shared GPU. Each ViT computation blocks all in-flight decode requests for its duration, effectively adding wait time to their decode phase. In the disaggregated case, ViT runs on the E worker and never occupies the PD GPU, so decode steps proceed without interruption. This is a transitional regime, where the disaggregation effect on TPOT changes from negligible to slightly positive. The actual TPOT benefit scales with the frequency of ViT invocations that overlap with a request’s decode phase, which in turn depends on the request arrival rate and ViT duration.

Heavy workload Under sustained high load with many concurrent requests at the GPU capacity limit, the ViT blocking effect becomes pronounced. Requests arrive frequently, and for heavy multi-modal workloads a single request can trigger several ViT calls on the shared GPU, which in turn interferes with the decode phase of all concurrently running requests, increasing their TPOT. Disaggregation eliminates ViT blocking entirely from the decode GPU, keeping TPOT low regardless of ViT workload intensity.

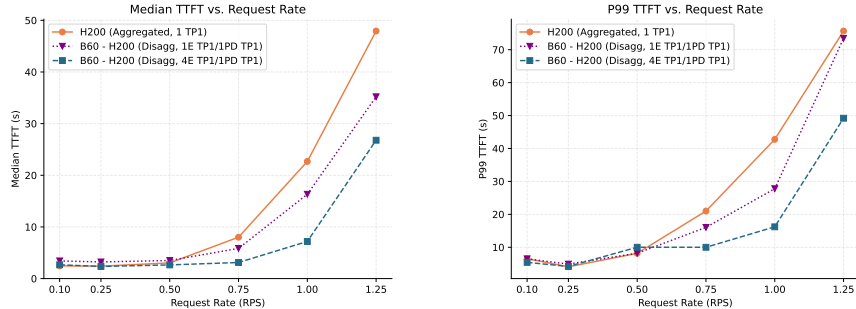


Figure 6: TTFT performance of Qwen3-VL-32B-FP8 under Dynamo E/PD disaggregation with 1 PD worker and 4 E workers, comparing an aggregated NVIDIA H200 baseline against a heterogeneous disaggregated deployment with four Intel Arc Pro B60 encoder workers connected to a single NVIDIA H200 PD worker (128 input tokens, 256 output tokens, and 20 images per request at 480p resolution).

TTFT vs. TPOT improvement It is important to emphasize that TTFT and TPOT have different primary reasons for improvement and different relative gains from disaggregation. For TTFT, the major part of the gain comes from reduced time that a request spends in the queue waiting for available GPU resources, which yields significant benefits. For TPOT, the gain is caused by the elimination of ViT interference and is relatively small.

4.2.3 Inter-Token Latency

Inter-Token Latency (ITL) records the time between each pair of consecutive tokens, yielding a full distribution rather than the per-request average that TPOT provides. Because TPOT smooths out variation within a request, it can mask transient stalls that degrade the user’s streaming experience. ITL exposes these stalls directly.

The workload-dependent behavior of ITL mirrors that of TPOT (Figure 4.2.1). The key difference is how interference manifests in each metric. TPOT absorbs latency spikes into a single average, diluting their visibility. ITL preserves them as outliers in the upper tail of the distribution, widening the gap between median and mean. Disaggregation reduces these spikes, producing a tighter ITL distribution with a lower mean while leaving the median largely unchanged.

4.3 Performance Charts

The workload-based analysis in Section 4.2 identified three operating regimes for multimodal serving systems: light, moderate, and heavy workloads. To further study the transition between these regimes, we evaluate TTFT as a

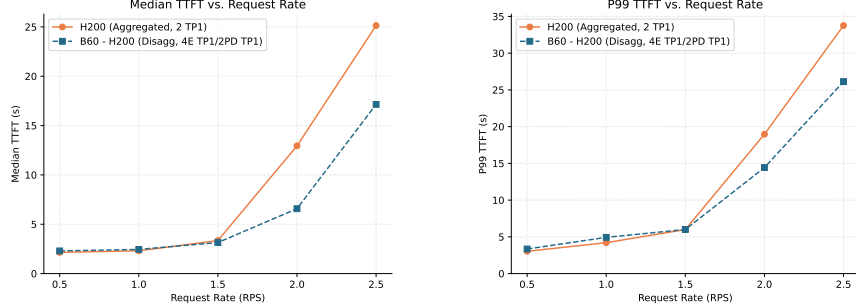


Figure 7: TTFT performance of Qwen3-VL-32B-FP8 under Dynamo E/PD disaggregation with 2 PD workers and 4 E workers, comparing an NVIDIA H200 baseline with two aggregated workers against a heterogeneous deployment with four Intel Arc Pro B60 encoder workers connected to two NVIDIA H200 PD workers (128 input tokens, 256 output tokens, and 20 images per request at 480p resolution).

function of request rate (RPS) under multiple aggregated and disaggregated system configurations.

Our performance evaluation uses the Qwen3-VL-32B-Instruct model, a dense multimodal LLM running in FP8 precision. Each request consists of 128 input tokens and 256 output tokens, and includes 20 images at 480×854 resolution. The workload is generated using `vllm.bench`, with images randomly sampled per request to eliminate any benefit from encoder caching or input reuse. Request arrivals follow a Poisson process with a controlled mean rate (RPS), producing a realistic and non-synchronized traffic pattern. Vision embeddings are transferred from E-workers to PD-workers using RDMA-backed NIXL communication in `nixl.write` mode. The experiments are conducted using Dynamo 1.0.0 with vLLM 0.19.1 as the inference engine backend.

Figure 6 evaluates a single PD-worker configuration and compares three deployment modes: an aggregated EPD worker, a disaggregated configuration with one E-worker and one PD-worker, and a disaggregated configuration with four E-workers connected to a single PD-worker. At low request rates (between 0.1 and 0.5 RPS), all configurations exhibit comparable TTFT, consistent with the light workload behavior discussed in Section 4.2. In this regime, disaggregation slightly increases latency due to communication and synchronization overhead. The single-E-worker configuration in particular performs worse than the aggregated baseline, indicating that the added transfer overhead is not offset by sufficient overlap opportunities at low concurrency.

As request rate increases, the aggregated configuration develops rapidly growing tail latency and queue buildup, while both disaggregated configurations maintain substantially lower TTFT. The four-E-worker configuration consistently achieves the best performance under increasing load, indicating that additional

encoder-side parallelism improves overlap and reduces contention on the latency-critical PD execution path. The single-E-worker configuration also improves over the aggregated baseline at higher request rates, but remains consistently worse than the four-E-worker configuration, suggesting diminishing returns from limited encoder capacity rather than an encoder-side bottleneck.

At the highest evaluated request rates, the aggregated configuration exhibits severe TTFT growth characteristic of the heavy workload behavior described earlier, while the disaggregated configurations delay the onset and magnitude of this degradation. At 1.0 RPS, for example, the four-E-worker configuration reduces median TTFT from 22.7 seconds to 7.2 seconds and p99 TTFT from 42.8 seconds to 16.2 seconds relative to the aggregated baseline.

To normalize the severe TTFT growth observed in the single-PD configuration, Figure 7 repeats the same experiment using two PD-workers and four E-workers. In this setting, the aggregated baseline consists of two monolithic EPD workers, increasing the effective system capacity and delaying the onset of saturation. As a result, moderate and heavy workload behavior does not become significant until request rates exceed approximately 1.5 RPS. Nevertheless, once contention begins increasing, the same qualitative trend reappears: the disaggregated configuration maintains substantially lower TTFT than the aggregated baseline at higher load levels. This improvement is driven by resource specialization, where encoder execution is isolated from the PD execution path, preventing ViT computation from amplifying latency in the critical prefill and decode stages.

Across both configurations, the results show a consistent trend: E/PD disaggregation does not primarily reduce TTFT under light load, but significantly improves scalability by shifting the onset of congestion to higher request rates and maintaining substantially lower TTFT in the moderate and heavy workload regimes.

5 Conclusion

We presented a heterogeneous E/PD-disaggregated serving architecture within the Dynamo framework for multimodal LLM inference. By separating visual encoding from the prefill and decode stages, the design enables independent scaling of encoder and decoder resources and allows ViT workloads to run on lower-cost accelerator hardware.

Our analysis shows that benefits are workload-dependent. Under light load, disaggregation provides little gain and may slightly increase TTFT due to communication overhead. Under moderate load, TTFT improves via overlap between embedding computation and prefill execution. Under heavy load, the dominant benefit shifts to queue reduction, as offloading ViT execution reduces contention on PD-workers and delays saturation.

Experiments further show that Intel Arc B60-based E-workers can sustain competitive TTFT despite significantly lower memory capacity and compute capability than NVIDIA H200 PD-workers, since encoder computation is a small fraction of the overall workload.

[TODO: Using normalized cost units, where an H200-class PD GPU is assigned a cost of 1.0 and a B60-class E-worker is assigned a cost of 0.025, the architecture allows TTFT improvements in the moderate and heavy workload regimes with only marginal increases in total system cost. This creates a favorable cost-performance trade-off in which latency under high load is primarily governed by the number of low-cost encoder workers rather than scaling of expensive decoder resources.]

6 Acknowledgments

The authors would like to thank Sandeep Maddipatla for his valuable feedback and helpful suggestions.

References

- [1] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Í nigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024.
- [2] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [3] Gursimran Singh, Xinglu Wang, Yifan Hu, Timothy Yu, Linzi Xing, Wei Jiang, Zhefeng Wang, Xiaolong Bai, Yi Li, et al. Efficiently serving large multimodal models using EPD disaggregation. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, Vancouver, Canada, 2025.
- [4] Fan Bai, Pai Peng, Zhengzhi Tang, Zhe Wang, Gong Chen, Xiang Lu, Yinuo Li, Huan Lin, Weizhe Lin, et al. EPD-Serve: A flexible multimodal EPD disaggregation inference serving system on ascend, 2026. arXiv eprint 2601.11590 <https://arxiv.org/abs/2601.11590>.
- [5] Multimodality Workstream @ vLLM. Encoder disaggregation for scalable multimodal model serving. vLLM Blog, December 2025. <https://vllm.ai/blog/vllm-epd>.
- [6] Tianyu Guo, Siyu Liu, Shangming Cai, and Wengang Zheng. EPD disaggregation: Elastic encoder scaling for vision-language models in SGLang. LMSYS Blog, January 2026. <https://www.lmsys.org/blog/2026-01-12-epd/>.

- [7] NVIDIA. Dynamo: A datacenter-scale distributed inference serving framework. GitHub, 2025. <https://github.com/ai-dynamo/dynamo>.
- [8] Anuj Kalia, Michael Kaminsky, and David G. Andersen. Design guidelines for high performance rdma systems. In *2016 USENIX Annual Technical Conference (USENIX ATC 16)*, pages 437–450. USENIX Association, 2016.